© The Author(s), under exclusive license to APress Media, LLC, part of Springer Nature 2025
S. K. Sahu, *Generative AI-Driven Application Development with Java*
https://doi.org/10.1007/979-8-8688-1609-3_3

3. The Transformer Saga—From Attention to Fine-Tuning

Satej Kumar Sahu[1]
(1) Berhampur, Odisha, India

The goal of this chapter is for you to understand the fundamental structure behind large language models (LLMs), which are transformers.

Introduction

Large language models have brought about a new paradigm shift in today's world. They have abilities to generate new content based on their foundational training on huge amounts of data. This data is multimodal, meaning in different formats like text, image, audio, videos, etc. This enables the LLMs to respond in different formats too. They are generally deep neural networks that have trained on massive amounts of data. Internally through this training the networks are able to understand the different patterns of the underlying data used for their training.

With this training, they are able to perform multiple tasks like content generation, question answering, summarization, and multiple other tasks. In this chapter, we will delve into the transformer architecture, which is the fundamental building block for LLMs and the history that led to it. It delves a bit into the theory part of LLMs and their technical history. If you choose to skip this chapter, this won't affect the skills needed to build generative AI applications using Java.

Importance of LLMs

If you have any knowledge of state-of-the-art natural language processing (NLP) techniques, you know they are focused on solving a particular problem like entity extraction or language translation, but not both. This is where LLMs are the current state-of-the-art content-trained models that are able to process content in multiple formats and

achieve multiple tasks like content generation, question answering, summarization, and language translation, all through the same single LLM. This is truly a revolution. LLMs are a general foundation model but capable of performing multiple kinds of tasks.

These models can be trained on domain-specific data to make them more oriented toward a particular task. They can also be trained or asked to reply in a certain style or perform activities in which they were not previously trained to by leveraging the art of prompt engineering, tuning the creativity level through temperature variation. As LLMs are evolving, their maximum token process capability has been improving, making them suitable for more content processing.

There are also techniques that you will learn about that create a fine-grained smaller model that can perform on a similar if a bit reduced level as compared to the larger models. It is still fast when loading, processing, and running on cheaper commodity central processing units (CPUs), if not graphical processing units (GPUs), thus bringing about the era of small language models. You will learn more about some of these in the coming sections.

It's interesting to use LLMs to build applications and see them performing these tasks with such ease for that previously you might have needed an expert team of specialized machine learning engineers. You might wonder how they work, though. The white paper "Attention is all you need" by Google is the foundation of all LLMs (**https://arxiv.org/abs/1706.03762**). It's an interesting read and recommended for everyone who wants to learn how LLMs work.

How Large Language Models Work

Think of a giant friendly teddy bear who has spent years listening to every bedtime story, song, and joke it could find. All those words and sentences are now squished inside its fluffy head like millions of colorful story blocks. When you ask the bear a question or start telling it a half-finished tale, it reaches into that big pile of blocks, remembers all the patterns it heard (like how "happily" often comes after "ever"), and quickly snaps together the next words that make the most sense. It isn't magic or real thinking; it's more like a super-fast guessing game based on everything it has heard before. Because the teddy bear has listened to *so* many stories, its guesses usually come out sounding just like something a person would say, whether you want a joke, an answer, or a brand-new bedtime adventure.

In essence, this is how a large language model functions. It predicts the probabilities of a sequence of words in its vocabulary. Typically, when a fragment of text is supplied, a

language model estimates the likelihood of each possible next word.

Let's review step-by-step how an LLM strings a sentence together, shown as a numbered sequence of mini decisions. We'll use the starter phrase **"The tallest animal on Earth is..."** to keep things concrete.

Here are the steps:

1. **Read the starter words.**
   The model turns each word—"The," "tallest," "animal," "on," "Earth," "is"—into numbers it can process (called *tokens*).
2. **Look back at everything so far.**
   Inside the model, an "attention" mechanism lets every word peek at every other word in the prefix, so it fully understands the topic (size ➤ animal ➤ Earth).
3. **Guess the next word.**
   Based on patterns it learned during training, the model produces a probability list for the very next token.
   Example distribution:
   1. "giraffe" → **0.65**
   2. "elephant" → 0.10
   3. "building" → 0.02
   4. "pizza" → 0.01
4. **Pick (or sample) the winner.**
   Most decoding methods choose the word with the highest probability ("giraffe") or sample from the top few choices to keep things varied.
5. **Attach the chosen word.**
   The sentence now reads: "The tallest animal on Earth is **giraffe**..."
6. **Loop with the new context.**
   The model feeds the updated sentence back into itself, recalculates attention, and guesses the *next* token.
   New distribution for token after "giraffe":
   1. "." (period) → **0.50**
   2. "the" → 0.25
   3. "and" → 0.05
7. **Decide whether to finish.**
   Selecting the period ends the sentence:
   "The tallest animal on Earth is giraffe."
8. **If not finished, keep going.**
   Had the model picked "the," it would continue the cycle—re-evaluating context, producing another probability list, selecting a word, and so on—until it reaches a

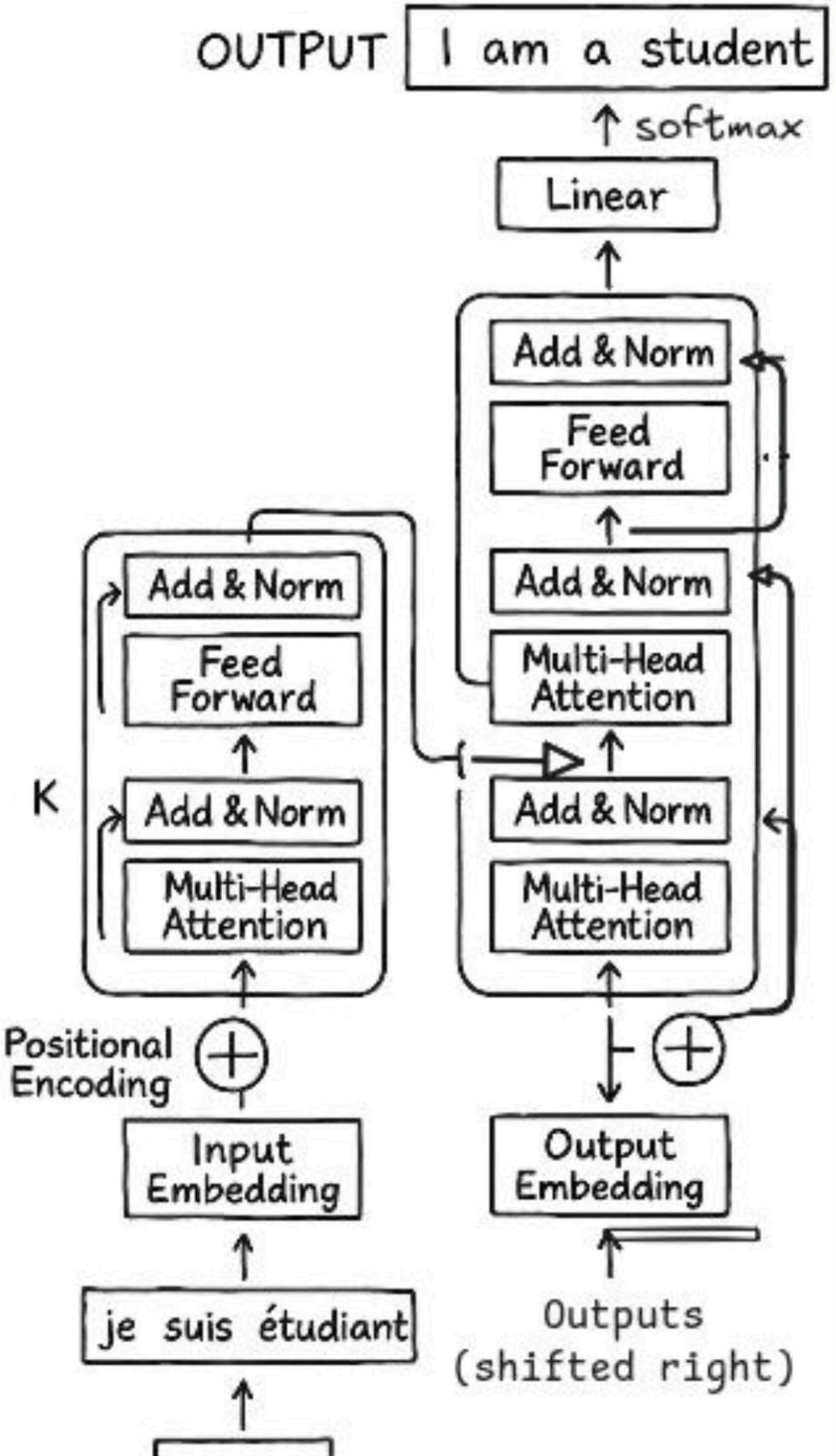
natural stopping point or a preset length limit.

In short, an LLM forms a sentence by **repeatedly** glancing at all the words it has so far, computing probabilities for what comes next, and committing to one token at a time until it decides the thought is complete.

Transformer

Imagine a big box of story blocks where each block is a word and, hovering over it, is a super-helpful robot called a transformer that builds sentences and stories. It spreads all the blocks out and looks at everything at once so it can spot which words belong together—just like noticing that "ice" usually sits near "cream." Then its magic sparkles of **attention** light up the words that matter most—so when it sees "dog," it points to "fluffy," "barks," and "tail," never confusing the pup with a "hot dog.". Next, the robot stacks layer upon layer of thinking caps, each one re-examining the sentence and adding a dose of extra cleverness, like building a taller and smarter LEGO tower. After all that thinking, it picks the next best word block and lines up the story so it sounds exactly right, whether it's finishing your sentence, answering a question, or inventing a bedtime tale. In short, the transformer is a super-fast story-building robot that sees all the words together, decides which ones matter most, and piles up many tiny thoughts to say something new and helpful.

The original transformer had two components: an encoder and a decoder, as shown in Figure 3-1. Imagine the transformer is asked to turn the Italian sentence "Il gatto dorme sul divano" ("The cat is sleeping on the couch") into English.


OUTPUT
I am a student
softmax
Linear
Add & Norm
Feed Forward
Add & Norm
Multi-Head Attention
Add & Norm
Multi-Head Attention
K
Add & Norm
Feed Forward
Add & Norm
Multi-Head Attention
Positional Encoding
Input Embedding
Output Embedding
je suis étudiant
Outputs (shifted right)

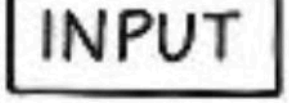


Figure 3-1 Original transformer architecture

Let's decode how it does the job step-by-step to understand the process:

1. **Encoder, the super listener:**
   First, the model "hears" the Italian words *Il gatto dorme sul divano* and changes each one into tiny number blocks it understands. All the encoder layers peek at the whole sentence at once and notice the important links—like that **"gatto"** (cat) is the thing doing **"dorme"** (sleeping). When it finishes listening, it isn't holding Italian words anymore—just a neat bundle that captures the meaning of "cat...sleeping... couch."
2. **Decoder, the careful talker:**
   Next, the decoder grabs that bundle of meaning and starts speaking English, one word at a time. It thinks, "Given what I know, which English word should come first?" It picks **"The."** Then it looks again at the meaning plus the word it just said and chooses the next one, **"cat,"** and then **"is," "sleeping," "on," "the,"** and **"couch."** After every new word, its little attention sparkles double-check that the sentence still matches the original idea.
3. **Layers, stacks of thinking caps:**
   Both the encoder and decoder consist of many identical layers—like stacking lots of thinking caps. Each cap adds a sprinkle of extra smarts, catching details such as the order of "on the couch" so the English comes out naturally.
4. **Why it works for any language:**
   Because the encoder stores only pure meaning and the decoder worries only about saying that meaning in the target language, you can swap Italian for Spanish, Japanese, or even emoji sentences—as long as the model has seen enough example pairs to learn from.

So, in simple words, the transformer **listens** to the whole Italian sentence, **packs** its meaning into a tidy bundle, and then **speaks** the English sentence one word at a time—always checking itself—until it proudly says: **"The cat is sleeping on the couch."**

Evolution of Transformers

Since the release of the 2017 paper *"Attention Is All You Need"*, transformers have undergone a rapid evolution, centered around three main trends: scaling up, becoming more efficient, and specializing for different use cases (see Figure 3-2). Much of this

innovation has been fueled by the rise of open-source models and the continued contributions from leading research labs like OpenAI.

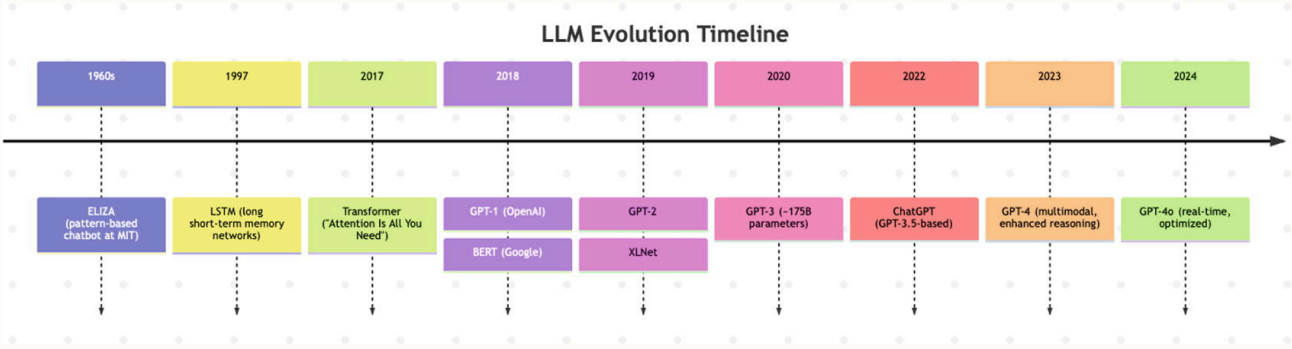


Figure 3-2 LLM evolution timeline

Among open-source models, Meta's LLaMA series has been a key driver. LLaMA 1 surprised the community by delivering near-GPT-3 performance with just 7 billion parameters. LLaMA 2 extended the context window to 4,096 tokens, improved quality with more training data, and introduced commercially usable models. With LLaMA 3, Meta scaled even further—offering up to 128,000-token context, better safety alignment, and quantized builds that can run on edge devices, making powerful language models more accessible and efficient.

Another standout is Mistral's Mixtral 8×7B, which uses a *Mixture of Experts* architecture. Instead of running the full 47 billion parameters, the model routes each token through only two expert networks, activating just 13 billion parameters per inference. This gives it top-tier performance—comparable to much larger models—while keeping latency low and making local deployment practical.

OpenAI has pushed the frontier in a different direction: explicit reasoning and planning. Its o1 model family is designed not just to generate fluent language, but to plan answers step-by-step using reinforcement learning. This approach has led to state-of-the-art results in math, science, and programming, where thoughtful, multistep reasoning matters more than surface-level fluency.

Another major contributor is DeepSeek, which takes a novel approach to self-improvement. Its DeepSeek-R1 model achieves competitive performance without relying heavily on large, labeled datasets. Instead, it uses a training technique called *group relative policy optimization*—allowing the model to evaluate and refine its own outputs iteratively. This self-training approach improves both quality and efficiency, pushing boundaries in reasoning and generalization with minimal human supervision.

Beyond these leaders, many organizations are contributing to a fast-growing open ecosystem. Releases like Alibaba's Qwen 1.5, 01. ai's Yi, xAI's Grok 3, NVIDIA's NVLM, and

Databricks' DBRX are extending context windows to hundreds of thousands—or even a million—tokens, adding support for multimodal inputs, and experimenting with compressed formats like 4-bit quantization to make models lighter and faster.

Together, these developments show a clear shift in direction. It's no longer just about building massive models; it's about building smarter, more efficient, and more accessible ones. Whether through better reasoning, smaller deployment footprints, or open licensing, the Transformer architecture continues to adapt and lead. With innovation now thriving not just in closed labs but across the open-source community, the future of transformers looks more collaborative, capable, and inclusive than ever.

Fine-Tuning LLMs

After an LLM is trained on massive amounts of general text during pretraining, it gains a solid grasp of language and can perform a variety of tasks through prompting. However, to make it truly useful, safe, or specialized for certain applications, it often needs fine-tuning—a second stage of training that adapts the base model to behave in more specific, controlled, or helpful ways. This section introduces the most common fine-tuning techniques and how they work.

Supervised Fine-Tuning

Supervised fine-tuning (SFT) is the most direct approach. In this method, the model is trained on carefully curated, labeled examples that demonstrate how it should behave. Each training example typically consists of a prompt and the correct response—such as a question and its answer, a document and its summary, or an instruction and the desired output. This teaches the model to follow specific tasks more accurately. SFT is especially useful for making the model follow instructions, generate consistent outputs, or understand domain-specific data. While it's less costly than pretraining, it still requires significant computing if applied across the entire model.

Dialogue and Instruction Tuning

Two common SFT use cases are instruction tuning, where the model learns to follow complex tasks like summarizing, writing, or coding from instructions, and dialogue tuning, where it's trained on conversational question-answer pairs. Dialogue tuning is essential for building chatbots or virtual assistants that can carry on multiturn conversations naturally and coherently.

Safety Tuning

To reduce harmful, biased, or toxic outputs, models can be fine-tuned using safety-focused datasets. This includes filtering harmful data, using human reviews, and incorporating explicit rules during training. Safety tuning ensures that the model avoids offensive or unethical content and behaves more responsibly in sensitive scenarios.

Reinforcement Learning from Human Feedback

Reinforcement Learning from Human Feedback (RLHF) is a powerful method used after SFT to make the model's responses align more closely with human preferences. Instead of only learning from good examples (like in SFT), RLHF also teaches the model what *not* to do by penalizing undesired responses. The process involves training a reward model using human feedback—where humans compare two model responses and indicate which one is better. The LLM is then further trained using this reward model and reinforcement learning techniques, improving qualities like helpfulness, truthfulness, and safety.

Parameter-Efficient Fine-Tuning

Full model fine-tuning (PEFT) is often computationally expensive. PEFT techniques reduce the cost by training only a small set of additional parameters while keeping the original model mostly frozen. These methods make it easier to adapt models to new tasks without retraining everything from scratch.

- **Adapter-based fine-tuning**: Small trainable modules (adapters) are added between layers of the model. Only these adapters are trained, significantly reducing memory and compute usage.
- **Low-rank adaptation (LoRA)**: Instead of updating the entire weight matrix, LoRA uses two small matrices to adjust the model. It's efficient, is easy to swap in and out, and can be combined with quantization for even more savings (e.g., QLoRA).
- **Soft prompting**: Instead of changing the model, this technique learns a few artificial prompt tokens (vectors) that steer the model's behavior. It's extremely light-weight—sometimes using just a handful of tokens—while enabling flexible task conditioning.

While full fine-tuning still offers the highest performance, PEFT techniques strike a strong balance between quality, speed, and cost, making them ideal for many real-world applications. Together, these methods allow developers to refine large models into more focused, friendly, and functional tools.

Inference Optimization Techniques

Once an LLM is trained and fine-tuned, optimizing its inference—how it generates outputs in real time—is crucial for making it efficient, fast, and cost-effective. This section summarizes key inference optimization methods, categorized by their approach and impact, and explains how they help reduce latency and resource consumption while maintaining or improving model quality.

Quantization

What it is: Reduces the precision of model weights and activations (e.g., from 32-bit to 8-bit), making computations lighter and faster.

Variants:

- **Post-training quantization**: Applied after training, mainly for inference.
- **Quantization-aware training (QAT)**: Incorporates quantization during training, allowing the model to adapt to the lower precision and recover quality.

  Why it helps: Speeds up inference and lowers memory usage, especially useful when balancing cost and quality.

Distillation

What it is: Trains a smaller model (student) to mimic a larger model (teacher), improving the student's performance without full-scale training.

Types of distillation:

- **Data distillation**: Uses a large model to generate synthetic data to train the smaller model.
- **Knowledge distillation**: Aligns the output distributions (e.g., token probabilities) of the student with those of the teacher for better efficiency.
- **On-policy distillation**: In reinforcement learning setups, the student learns from the teacher's evaluations of its own outputs.

  Why it helps: Allows deployment of smaller, faster models while retaining most of the accuracy of larger models.

Output-Preserving Optimizations

What it is: Techniques that reduce inference time without altering the model's output in any way.

Key methods:

- **FlashAttention**: Rewrites attention computation to reduce memory traffic between slower and faster memory in GPUs/TPUs. Achieves 2–4× speedups without changing the result.
- **Prefix Caching**: Stores intermediate computation (KV cache) from previous inputs so they don't have to be recalculated in multiturn conversations or repeated prompts.
  Why it helps: These methods are risk-free optimizations—perfect for reducing latency without any trade-off in output quality.

Speculative Decoding

What it is: Uses a smaller model (drafter) to predict several tokens ahead and then has the main model verify them in parallel.

How it works: The main model doesn't blindly trust the drafter—it checks the predictions token by token and accepts only the verified outputs.

Why it helps: Significantly speeds up the slow, step-by-step decoding phase of LLMs without sacrificing output correctness.

Batching and Parallelization

What it is: Classic software optimization strategies adapted for LLMs to boost throughput.

- **Batching**: Combines multiple requests so they can be processed simultaneously on the same hardware. Particularly effective during the decode phase.
- **Parallelization**: Distributes the model across multiple machines or hardware cores using techniques like tensor parallelism, pipeline parallelism, or sequence parallelism.
  Why it helps: Maximizes hardware usage and accelerates response time, though careful design is needed to manage communication overhead between devices.

Category	Key Benefit	Typical Use Case
Quantization	Smaller, faster models	Edge deployment, memory-constrained environments
Distillation	Compresses model size with minimal quality loss	Lightweight chatbots, mobile apps
Output-Preserving	Fast, safe latency reduction	Enterprise LLMs, chat apps, document Q&A

Category	Key Benefit	Typical Use Case
Speculative Decoding	Parallelizes sequential steps	Real-time assistants, search
Batching & Parallelization	Throughput and latency improvements	High-volume API servers, model serving platforms

Together, these techniques allow LLMs to respond faster, run more efficiently, and scale across different hardware and user needs—all without compromising the integrity of the model's outputs.

Conclusion

In this chapter, you journeyed through the foundation of today's most advanced AI systems—transformers, the architecture at the heart of all large language models . You explored their structure, the groundbreaking innovation of self-attention, and how they evolved into the scalable, multimodal giants used today. From understanding how transformers predict and generate language to tracing the rapid evolution of models like GPT, LLaMA, Gemini, and Mixtral, we saw how the field has shifted from building just large models to making them *efficient*, *safe*, and *adaptable*.

You also examined the vital role of fine-tuning, where pretrained models are tailored to follow instructions, engage in dialogue, or behave more safely—often using advanced techniques like RLHF or lightweight methods like LoRA and soft prompting. Finally, you looked at how inference optimization ensures these powerful models can run affordably and in real time, whether on cloud servers or mobile devices.

In essence, transformers aren't just a technical marvel—they are the engine powering the current AI revolution. Understanding their mechanics, history, and practical adaptations equips you with the context to better design, optimize, or simply appreciate the intelligence behind today's generative AI. While you don't need to master these details to build applications using Java, having this foundational knowledge makes you a more informed and capable developer in this rapidly evolving landscape.

The next chapter covers self-hosting LLMs using Ollama.

‹ 2. First Contact: “Hello, LLM” with Spring Boot

4. Bring Your Own Model: Self-Hosting with Ollama ›